T.C.

GAZİ ÜNİVERSİTESİ

FEN FAKÜLTESİ

MATEMATİK BÖLÜMÜ

ARAŞTIRMA PROJESİ 2

LİNEER OLMAYAN DENKLEMLERİN NÜMERİK ÇÖZÜMLERİ

HAZIRLAYAN

RASİM KAYA

200420094

DANIŞMANI

Prof. Dr. MEDİHA ÖRKCÜ

Haziran 2024

İÇİNDEKİLER

- Giriş 1

- Lineer Olmayan Denklemlerin Nümerik Çözümleri.....3

1.1 Teoremler.....3
2.1 Aralık Yarılama Yöntemi.....4
2.2 Python Kodu.....5
2.3 Python Çıktısı …….....6
3.1 Sabit Nokta İterasyonu...…….....9
3.2 Python Kodu.....10
3.3 Python Çıktısı …….....11
4.1 Newton Raphson Methodu ….....14
4.2 Python Kodu.....15
4.3 Python Çıktısı 16

GİRİŞ

LİNEER OLMAYAN DENKLEMLERİN NÜMERİK ÇÖZÜMLERİ

Örnek:

$$\boldsymbol{x^3 + sinx - 1 = 0, e^x + x^2 - 5 = 0, lnx - 2x + 1 = 0...}$$

Sayısal bir yöntem oluşturabilmemiz için kökün varlığından, yani varlığını bildiğimiz bir aralığa ihtiyacımız var. Bu yüzden bazı teoremleri bilmeliyiz, bilmemiz gereken teoremler başlıca;

TEOREM I (ROLLE TEOREMİ)

Bir f fonksiyonu [a,b] aralığında sürekli ve (a,b) aralığında türevlenebilir olsun.

$Eğer\ f(a) = f(b)\ ise\ f'(c) = 0$

Olacak şekilde bir c ∈ (a,b) vardır.

TEOREM II (ORTALAMA DEĞER TEOREMİ)

f fonksiyonu [a,b] aralığında sürekli ve (a,b) aralığında türevlenebilir olsun.

$f'(c) = [f(b) - f(a)] / (b - a)$

Olacak şekilde c ∈ (a,b) vardır**.**

TEOREM III (ARA DEĞER TEOREMİ)

f fonksiyonu [a,b] aralığında sürekli olsun.

Eğer bir K sayısı f(a) ile f(b)'nin arasında ise f(c)=K olacak şekilde bir K ∈ (a,b) vardır.

TEOREM IV (BOLZANO TEOREMİ)

f, [a,b]'de sürekli olsun.

Eğer f(a). f(b) < 0 ise f(c) = 0 olacak şekilde bir c ∈ (a,b) vardır.

ÖRNEK:

$f(x) = x^5 - 2x + 1\ verilsin.$

$f(-2) = -32 + 4 + 1 = -27 < 0$

$f(0) = 0 + 0 + 1 = 1 > 0$

$f(a).f(b) = f(0).f(-2) < 0$

O halde f(c) = 0 olacak şekilde c ∈ (-2,0) vardır.

2.1 ARALIK YARILAMA YÖNTEMİ

Bu yöntem aradeğer teoremi kullanılarak kökün olduğu bir aralık seçimi üzerine duyarlıdır. Diyelim ki f(x) verilsin ve [a,b] aralığında bir kökü olsun.

Yani f(a). f(b) < 0'dır.

a = a_1 ve b = b_1 olsun, o zaman [a_1,b_1] = [a,b] olur, sıradaki aralığı bulmak için;

P_1 = (a+b) / 2 hesaplanır.

Eğer f(a) . f(P_1) < 0 ise a_1 = a_1 ve b_2 = p_1

Eğer f(b) . f(P_1) < 0 ise a_2 = p_1 ve b_2 = b_1

İterasyon bu şekilde devam ettirilir ve;

$[a_1, b_1]$, $[a_2, b_2]$, $[a_3, b_3]$, $[a_4, b_4]$, ... , $[a_n, b_n]$ aralıkları elde edilir.

Eğer f'nin bir kökü P ise $n \longmapsto \infty \, lim(a_n) = n \longmapsto \infty \, lim(b_n) = P$, ayrıca $P_1, P_2, P_3, \ldots, P_n$ dizisi olsun.

O halde $n \longmapsto \infty \, lim(P_n) = P$

Bu şekilde oluşan aralık yarılamaları yapmak yerine

p_n dizisinin $| P_n - P| \leq (b - a) / 2^n$ şeklinde direkt kaç kez iterasyon yapmamız gerektiğini öğrenebiliriz.

ÖRNEK:

Pyhton yazılım dilinde aşağıdaki görselde verilen örnekte; kullanıcının herhangi bir denklemi girdiği, ve benzer şekilde kullanıcıdan [a,b] kapalı aralığını belirlemesini isteyen, ve verilen bu değerlere göre bu aralıkta denklemin kökünün varlığını araştıran, daha sonrasında ise eğer bu aralıkta kök var ise yeni aralığı belirleyen, ve kaç kez iterasyon yapmamız gerektiğini istediği takdirde o kadar iterasyon yapan bir program gösteriliyor.

2.2 Pyhton kodu:

```
import math
from math import sin, exp

def aralik_yarilama(denklem, a, b, hata=None, max_iterasyon=None):
    if hata is None:
        max_iterasyon = int(input("Kaç iterasyon yapılmasını istersiniz?: "))  # Kullanıcıdan
iterasyon sayısını alalım

    if denklem(a) * denklem(b) > 0:
        raise ValueError("Başlangıç değerleri arasında kök yok.")
    print(f"\nf({a}) = {denklem(a)} , f({b}) = {denklem(b)}\n")
    print(f"Girilen [{a}, {b}] aralığında kök vardır. Çünkü f({a}) * f({b}) < 0\n")

    iterasyon = 0
    while True:
        c = (a + b) / 2
        fa, fb, fc = denklem(a), denklem(b), denklem(c)

        print(f"{iterasyon + 1}. İterasyon:")
        print(f"a = {a}, f(a) = {fa}")
        print(f"b = {b}, f(b) = {fb}")
        print(f"c = (a + b) / 2 = ({a} + {b}) / 2 = {c}, f(c) = {fc}")
        hata_degeri = abs(b - a) / 2 ** (iterasyon + 1)
        print(f"Hata değeri = (|b - a| / 2^(iterasyon + 1)): {hata_degeri}")

        if fc == 0 or (hata is not None and abs(b - a) / 2** (iterasyon + 1) < hata):
            print(f"Hata değeri: {hata}")
            print("Kök bulunmuştur.")
            return c, iterasyon + 1

        if fc * fa < 0:
            b = c
            print(f"f(c) * f(a) < 0 olduğundan yeni aralığımız [{a}, {b}]\n")
        elif fc * fb < 0:
            a = c
            print(f"f(c) * f(b) < 0 olduğundan yeni aralığımız [{a}, {b}]\n")

        iterasyon += 1
        if max_iterasyon is not None and iterasyon >= max_iterasyon:
            print(f"Maximum iterasyon sayısına ulaşıldı, kök bulunamadı. Son bulunan değer kök olarak
kabul edilecek.")
            return c, iterasyon + 1

    print("Devam etmesi gerekiyor.")
    return c, iterasyon + 1


hata_secim = input("Hata değeri kullanmak istiyor musunuz? (1: Evet, 2: Hayır): ")

if hata_secim == '1':
    hata = input("Hata değeri giriniz: ")
    hata = eval(hata)
    if hata < 0:
        raise ValueError("Hata değeri negatif olamaz.")
elif hata_secim == '2':
    hata = None
else:
    raise ValueError("Geçersiz seçim.")

a = float(input("[a,b] Girilecek olan aralık olmak üzere sırasıyla; \n(a): "))
b = float(input("(b): "))

# Kullanıcıdan denklemi isteyelim
denklem_str = input("Denklemi giriniz (örneğin: 3*x + sin(x) - exp(x)): ")

# Kullanıcıdan alınan denklem stringini değerlendirmek için eval() fonksiyonunu kullanalım
def denklem(x):
    return eval(denklem_str, {'x': x, 'sin': sin, 'exp': exp})

try:
    kok, iterasyon = aralik_yarilama(denklem, a, b, hata=hata)
    print(f"Sonuç: {kok}")
    print(f"Iterasyon sayısı: {iterasyon}")
```

```
except ValueError as hata:
    print(hata)
except Exception as hata:
    print("Beklenmeyen bir hata oluştu.")
    print(hata)
```

2.3 Pyhton Çıktısı:

```
Hata değeri kullanmak istiyor musunuz? (1: Evet, 2: Hayır): 1
Hata değeri giriniz: 0.01
[a,b] Girilecek olan aralık olmak üzere sırasıyla;
(a): 0
(b): 1
Denklemi giriniz (örneğin: 3*x + sin(x) - exp(x)): x**3-x**2+14*x-6

f(0.0) = -6.0 , f(1.0) = 8.0

Girilen [0.0, 1.0] aralığında kök vardır. Çünkü f(0.0) * f(1.0) < 0

1. İterasyon:
a = 0.0, f(a) = -6.0
b = 1.0, f(b) = 8.0
c = (a + b) / 2 = (0.0 + 1.0) / 2 = 0.5, f(c) = 0.875
Hata değeri = (|b - a| / 2^(iterasyon + 1)): 0.5
f(c) * f(a) < 0 olduğundan yeni aralığımız [0.0, 0.5]

2. İterasyon:
a = 0.0, f(a) = -6.0
b = 0.5, f(b) = 0.875
c = (a + b) / 2 = (0.0 + 0.5) / 2 = 0.25, f(c) = -2.546875
Hata değeri = (|b - a| / 2^(iterasyon + 1)): 0.125
f(c) * f(b) < 0 olduğundan yeni aralığımız [0.25, 0.5]

3. İterasyon:
a = 0.25, f(a) = -2.546875
b = 0.5, f(b) = 0.875
c = (a + b) / 2 = (0.25 + 0.5) / 2 = 0.375, f(c) = -0.837890625
Hata değeri = (|b - a| / 2^(iterasyon + 1)): 0.03125
f(c) * f(b) < 0 olduğundan yeni aralığımız [0.375, 0.5]

4. İterasyon:
a = 0.375, f(a) = -0.837890625
b = 0.5, f(b) = 0.875
```

```
c = (a + b) / 2 = (0.375 + 0.5) / 2 = 0.4375, f(c) = 0.017333984375
Hata değeri = (|b - a| / 2^(iterasyon + 1)): 0.0078125
Hata değeri: 0.01
Kök bulunmuştur.
Sonuç: 0.4375
Iterasyon sayısı: 4
```

biception.py

```python
import math
from math import sin, exp

def aralik_yarilama(denklem, a, b, hata=None, max_iterasyon=None):
    if hata is None:
        max_iterasyon = int(input("Kaç iterasyon yapılmasını istersiniz?: "))  # Kullanıcıdan iterasyon sayısını alalım

    if denklem(a) * denklem(b) > 0:
        raise ValueError("Başlangıç değerleri arasında kök yok.")
    print(f"\nf({a}) = {denklem(a)}, f({b}) = {denklem(b)}\n")
    print(f"Girilen [{a}, {b}] aralığında kök vardır. Çünkü f({a}) * f({b}) < 0\n")

    iterasyon = 0
    while True:
        c = (a + b) / 2
        fa = denklem(a)
        fb = denklem(b)
        fc = denklem(c)

        print(f"{iterasyon + 1}. İterasyon:")
        print(f"a = {a}, f(a) = {fa}")
        print(f"b = {b}, f(b) = {fb}")
        print(f"c = (a + b) / 2 = ({a} + {b}) / 2 = {c}, f(c) = {fc}")
        hata_degeri = abs(b - a) / 2 ** (iterasyon + 1)
        print(f"Hata değeri = (|b - a| / 2^(iterasyon + 1)): {hata_degeri}")

        if fc == 0 or (hata is not None and abs(b - a) / 2** (iterasyon + 1) < hata):
            print(f"Hata değeri: {hata}")
            print("Kök bulunmuştur.")
            return c, iterasyon + 1

        if fc * fa < 0:
            b = c
            print(f"f(c) * f(a) < 0 olduğundan yeni aralığımız [{a}, {b}]\n")
        elif fc * fb < 0:
            a = c
            print(f"f(c) * f(b) < 0 olduğundan yeni aralığımız [{a}, {b}]\n")

        iterasyon += 1
        if max_iterasyon is not None and iterasyon >= max_iterasyon:
            print(f"Maximum iterasyon sayısına ulaşıldı, kök bulunamadı. Son bulunan değer kök olarak kabul edilecek.")
            return c, iterasyon + 1

    print("Devam etmesi gerekiyor.")
    return c, iterasyon + 1
```

biception.py

```python
            a = c
            print(f"f(c) * f(b) < 0 olduğundan yeni aralığımız [{a}, {b}]\n")

        iterasyon += 1
        if max_iterasyon is not None and iterasyon >= max_iterasyon:
            print(f"Maximum iterasyon sayısına ulaşıldı, kök bulunamadı. Son bulunan değer kök olarak kabul edilecek.")
            return c, iterasyon + 1

    print("Devam etmesi gerekiyor.")
    return c, iterasyon + 1


hata_secim = input("Hata değeri kullanmak istiyor musunuz? (1: Evet, 2: Hayır): ")

if hata_secim == '1':
    hata = input("Hata değeri giriniz: ")
    hata = eval(hata)
    if hata < 0:
        raise ValueError("Hata değeri negatif olamaz.")
elif hata_secim == '2':
    hata = None
else:
    raise ValueError("Geçersiz seçim.")

a = float(input("[a,b] Girilecek olan aralık olmak üzere sırasıyla; \n(a): "))
b = float(input("(b): "))

# Kullanıcıdan denklemi isteyelim
denklem_str = input("Denklemi giriniz (örneğin: 3*x + sin(x) - exp(x)): ")

# Kullanıcıdan alınan denklem stringini değerlendirmek için eval() fonksiyonunu kullanalım
def denklem(x):
    return eval(denklem_str, {'x': x, 'sin': sin, 'exp': exp})

try:
    kok, iterasyon = aralik_yarilama(denklem, a, b, hata=hata)
    print(f"Sonuç: {kok}")
    print(f"Iterasyon sayısı: {iterasyon}")
except ValueError as hata:
    print(hata)
except Exception as hata:
    print("Beklenmeyen bir hata oluştu.")
    print(hata)
```

biception

```
C:\Users\xrasi\PycharmProjects\numerikanaliz\venv\Scripts\python.exe C:\Users\xrasi\PycharmProjects\numerikanaliz\biception.py
Hata değeri kullanmak istiyor musunuz? (1: Evet, 2: Hayır): 1
Hata değeri giriniz: 0.01
[a,b] Girilecek olan aralık olmak üzere sırasıyla;
(a): 0
(b): 1
Denklemi giriniz (örneğin: 3*x + sin(x) - exp(x)): x**3-x**2+14*x-6

f(0.0) = -6.0 , f(1.0) = 8.0

Girilen [0.0, 1.0] aralığında kök vardır. Çünkü f(0.0) * f(1.0) < 0

1. İterasyon:
a = 0.0, f(a) = -6.0
b = 1.0, f(b) = 8.0
c = (a + b) / 2 = (0.0 + 1.0) / 2 = 0.5, f(c) = 0.875
Hata değeri = (|b - a| / 2^(iterasyon + 1)): 0.5
f(c) * f(a) < 0 olduğundan yeni aralığımız [0.0, 0.5]

2. İterasyon:
a = 0.0, f(a) = -6.0
b = 0.5, f(b) = 0.875
c = (a + b) / 2 = (0.0 + 0.5) / 2 = 0.25, f(c) = -2.546875
Hata değeri = (|b - a| / 2^(iterasyon + 1)): 0.125
f(c) * f(b) < 0 olduğundan yeni aralığımız [0.25, 0.5]

3. İterasyon:
a = 0.25, f(a) = -2.546875
b = 0.5, f(b) = 0.875
c = (a + b) / 2 = (0.25 + 0.5) / 2 = 0.375, f(c) = -0.837890625
Hata değeri = (|b - a| / 2^(iterasyon + 1)): 0.03125
f(c) * f(b) < 0 olduğundan yeni aralığımız [0.375, 0.5]

4. İterasyon:
a = 0.375, f(a) = -0.837890625
b = 0.5, f(b) = 0.875
c = (a + b) / 2 = (0.375 + 0.5) / 2 = 0.4375, f(c) = 0.017333984375
Hata değeri = (|b - a| / 2^(iterasyon + 1)): 0.0078125
Hata değeri: 0.01
Kök bulunmuştur.
Sonuç: 0.4375
Iterasyon sayısı: 4

Process finished with exit code 0
```

3.1 SABİT NOKTA İTERASYONU

Bir g fonksiyonu için g(P) = P oluyor ise p'ye g'nin sabit noktası denir. Peki bunu fonksiyonların

köklerini bulmakta nasıl kullanacağız?

Örneğin;

$f(x) = x^3 - x + 1$ olsun.

$f(x) = 0$ çözmek demek $x^3 - x + 1 = 0 \Longrightarrow x^3 + 1 = x$ ve $g(x) = x^3 + 1$ dersek

$g(x) = x$ olur, $f(x) = 0$'ın kökü g(x)'in sabit noktası olur.

Bu iterasyon için öncelikle Sabit Nokta Teoremini açıklamamız gerek, öyle ki;

Bir g fonksiyonu [a,b]'da sürekli ve g: [a,b] $\longmapsto$ [a,b] tanımlansın. Bu durumda g'nin [a,b] aralığında bir sabit noktası vardır.

Eğer g, (a,b)'de türevli ve $|g'(x)| \leq k < 1$ olacak şekilde bir k sayısı varsa g'nin sabit noktaları tektir.

Bu k noktası var olsun, o halde;

❁ Sabit nokta iterasyonu (P_n) köke yakınsar, yani g(p) = p ise $P_n \longmapsto P$

❁❁ $|P_n - P_{n-1}| \leq k^{n-1} \cdot |P_1 - P_0|$ veya $|P_n - P_{n-1}| \leq k^{n-1} \cdot \text{maks}(P_0 - a\ ,\ b - P_0)$

❁❁❁ $|P_n - P| \leq k^{n-1}/(1-k) \cdot |P_1 - P_0|$

durumları sağlanır.

ÖRNEK:

Pyhton yazılım dilinde aşağıdaki görselde verilen örnekte; kullanıcının herhangi girdiği bir fonksiyonu ve benzer şekilde kullanıcıdan başlangıç değerini ve iterasyon sayısını isteyen, ve verilen bu değerlere göre sabit nokta iterasyonu yapan bir program gösteriliyor.

3.2 Python Kodu:

```
import math

def sabit_nokta_iterasyonu():
    # Kullanıcıdan fonksiyon girişi almak için bir fonksiyon
    def girdi_al():
        while True:
            try:
   fonksiyon_str = input("Lütfen iterasyon için kullanılacak fonksiyonu girin (örn: 'x**2 - 2*x - 3'):
")
                # Güvenlik amacıyla kullanılabilir fonksiyonlar sınırlanır
                ortam = {"_builtins_": None, "math": math}
                f = lambda x: eval(kod, {"x": x, **ortam})
                break
            except Exception as e:
                print(f"Hata: {e}. Lütfen fonksiyon ifadenizi kontrol edin ve tekrar deneyin.")
        return f

    # Başlangıç değeri girişi almak için bir fonksiyon
    def baslangic_degeri_girdisi():
        while True:
            try:
                baslangic_degeri = float(input("Lütfen başlangıç değerini girin: "))
                break
            except ValueError:
                print("Geçersiz giriş. Lütfen bir sayı giriniz.")
        return baslangic_degeri

    # İterasyon sayısı girişi almak için bir fonksiyon
    def iterasyon_sayisi_girdisi():
        while True:
            try:
                iterasyon_sayisi = int(input("Lütfen iterasyon sayısını girin: "))
                if iterasyon_sayisi <= 0:
                    print("İterasyon sayısı sıfırdan büyük olmalıdır.")
                    continue
                break
            except ValueError:
                print("Geçersiz giriş. Lütfen bir tam sayı giriniz.")
        return iterasyon_sayisi

    # Sabit nokta iterasyonunu hesaplamak için bir fonksiyon
    def sabit_nokta_iterasyonu_hesapla(f, baslangic_degeri, iterasyon_sayisi):
        p = baslangic_degeri
        print(f"\nBaşlangıç değeri: {p}")
        for i in range(iterasyon_sayisi):
            p_sonraki = f(p)
            print(f"{i + 1}. iterasyon: p = {p_sonraki}")
            p = p_sonraki

    # Fonksiyon girdisi alınır
    f = girdi_al()
    # Başlangıç değeri girdisi alınır
    baslangic_degeri = baslangic_degeri_girdisi()
    # İterasyon sayısı girdisi alınır
    iterasyon_sayisi = iterasyon_sayisi_girdisi()

    # Sabit nokta iterasyonu hesaplanır
    sabit_nokta_iterasyonu_hesapla(f, baslangic_degeri, iterasyon_sayisi)

# Programı çalıştırın
sabit_nokta_iterasyonu()
```

3.3 Pyhton Çıktısı:

Lütfen iterasyon için kullanılacak fonksiyonu girin (örn: 'x**2 - 2*x - 3'): math.exp(-x)

Lütfen başlangıç değerini girin: 0

Lütfen iterasyon sayısını girin: 25

```
Başlangıç değeri: 0.0
1. iterasyon: p = 1.0
2. iterasyon: p = 0.36787944117144233
3. iterasyon: p = 0.6922006275553464
4. iterasyon: p = 0.5004735005636368
5. iterasyon: p = 0.6062435350855974
6. iterasyon: p = 0.545395785975027
7. iterasyon: p = 0.5796123355033789
8. iterasyon: p = 0.5601154613610891
9. iterasyon: p = 0.571143115080177
10. iterasyon: p = 0.5648793473910495
11. iterasyon: p = 0.5684287250290607
12. iterasyon: p = 0.5664147331468833
13. iterasyon: p = 0.5675566373282834
14. iterasyon: p = 0.5669089119214953
15. iterasyon: p = 0.5672762321755696
16. iterasyon: p = 0.5670678983907884
17. iterasyon: p = 0.567186050099357
18. iterasyon: p = 0.5671190400572149
19. iterasyon: p = 0.5671570440012975
20. iterasyon: p = 0.5671354902062784
21. iterasyon: p = 0.5671477142601192
22. iterasyon: p = 0.567140781458298
23. iterasyon: p = 0.56714471334657
24. iterasyon: p = 0.5671424834013071
25. iterasyon: p = 0.5671437480994115
```

Process finished with exit code 0

```python
import math

def sabit_nokta_iterasyonu():
    # Kullanıcıdan fonksiyon girisi almak için bir fonksiyon
    def girdi_al():
        while True:
            try:
                fonksiyon_str = input("Lütfen iterasyon için kullanılacak fonksiyonu girin (örn: 'x**2 - 2*x - 3'): ")
                kod = compile(fonksiyon_str, "<string>", "eval")
                # Güvenlik amacıyla kullanılabilir fonksiyonlar sınırlanır
                ortam = {"__builtins__": None, "math": math}
                f = lambda x: eval(kod, {"x": x, **ortam})
                break
            except Exception as e:
                print(f"Hata: {e}. Lütfen fonksiyon ifadenizi kontrol edin ve tekrar deneyin.")
        return f

    # Başlangıç değeri girisi almak için bir fonksiyon
    def baslangic_degeri_girdisi():
        while True:
            try:
                baslangic_degeri = float(input("Lütfen başlangıç değerini girin: "))
                break
            except ValueError:
                print("Geçersiz giris. Lütfen bir sayı giriniz.")
        return baslangic_degeri

    # İterasyon sayısı girisi almak için bir fonksiyon
    def iterasyon_sayisi_girdisi():
        while True:
            try:
                iterasyon_sayisi = int(input("Lütfen iterasyon sayısını girin: "))
                if iterasyon_sayisi <= 0:
                    print("İterasyon sayısı sıfırdan büyük olmalıdır.")
                    continue
                break
            except ValueError:
                print("Geçersiz giris. Lütfen bir tam sayı giriniz.")
        return iterasyon_sayisi

    # Sabit nokta iterasyonunu hesaplamak için bir fonksiyon
    def sabit_nokta_iterasyonu_hesapla(f, baslangic_degeri, iterasyon_sayisi):
        p = baslangic_degeri
        print(f"\nBaşlangıç değeri: {p}")
        for i in range(iterasyon_sayisi):
            p_sonraki = f(p)
            print(f"{i + 1}. iterasyon: p = {p_sonraki}")
            p = p_sonraki

    # Fonksiyon girdisi alınır
    f = girdi_al()
    # Başlangıç değeri girdisi alınır
    baslangic_degeri = baslangic_degeri_girdisi()
    # İterasyon sayısı girdisi alınır
    iterasyon_sayisi = iterasyon_sayisi_girdisi()

    # Sabit nokta iterasyonu hesaplanır
    sabit_nokta_iterasyonu_hesapla(f, baslangic_degeri, iterasyon_sayisi)

# Programı çalıştırın
sabit_nokta_iterasyonu()
```

```
C:\Users\xrasi\PycharmProjects\numerikanaliz\venv\Scripts\python.exe C:\Users\xrasi\PycharmProjects\numerikanaliz\sabit.py
Lütfen iterasyon için kullanılacak fonksiyonu girin (örn: 'x**2 - 2*x - 3'): math.exp(-x)
Lütfen başlangıç değerini girin: 0
Lütfen iterasyon sayısını girin: 25

Başlangıç değeri: 0.0
1. iterasyon: p = 1.0
2. iterasyon: p = 0.36787944117144233
3. iterasyon: p = 0.6922006275553464
4. iterasyon: p = 0.5004735005636368
5. iterasyon: p = 0.6062435350855974
6. iterasyon: p = 0.545395785975027
7. iterasyon: p = 0.5796123355033789
8. iterasyon: p = 0.5601154613610891
9. iterasyon: p = 0.571143115080177
10. iterasyon: p = 0.5648793473910495
11. iterasyon: p = 0.5684287250290607
12. iterasyon: p = 0.5664147331468833
13. iterasyon: p = 0.5675566373282834
14. iterasyon: p = 0.5669089119214953
15. iterasyon: p = 0.5672762321755696
16. iterasyon: p = 0.5670678983907884
17. iterasyon: p = 0.567186050099357
18. iterasyon: p = 0.5671190400572149
19. iterasyon: p = 0.5671570440012975
20. iterasyon: p = 0.5671354902062784
21. iterasyon: p = 0.5671477142601192
22. iterasyon: p = 0.567140781458298
23. iterasyon: p = 0.56714471334657
24. iterasyon: p = 0.5671424834013071
25. iterasyon: p = 0.5671437480994115

Process finished with exit code 0
```

4.1 NEWTON – RAPHSON YÖNTEMİ

Bir f fonksiyonu 2. mertebeden sürekli türevli olsun, yani $f \in C^2[a, b]$. Bir $\bar{x}$ noktası civarında

Taylor seri açılımına bakalım;

$$f(x) = f(\bar{x}) + (x - \bar{x}) . f'(x) + [(x - \bar{x})^2 . f''(x)]/2!$$

diyelim ki bir p noktası için $f(P) = 0$ olsun. ve P civarında seri açılımı;

$f(P) = 0 = f(\bar{x}) + (P - \bar{x}).f'(\bar{x}) + R$ şeklinde olsun.

O halde

$P \approx \bar{x} - f(\bar{x}) / f'(\bar{x})$ eşitliğinden yaklaşık iterasyon oluşturulabilir.

Bu iterasyon ise;

$P_{n+1} = P_n - [f(P_n)]/f'(P_n)$ şeklinde olur.

Eğer herhangi bir p değeri için $f(P) = 0$ ve $f'(P) \neq 0$ ise;

$P_0 \in [P - \beta , P + \beta]$ olacak şekilde bir $\beta > 0$ vardır ve P_0 başlangıç değeri ile N-R metodu köke

yakınsar.

ÖRNEK:

Pyhton yazılım dilinde aşağıdaki görselde verilen örnekte; kullanıcının herhangi girdiği bir fonksiyonu ve benzer şekilde kullanıcıdan başlangıç değerini ve iterasyon sayısını isteyen, ve verilen bu değerlere göre N-R iterasyonu yapan bir program gösteriliyor

4.2 Pyhton Kodu:

```
from sympy import symbols, diff, sin, exp, cos

def newton_raphson(fonksiyon, fonksiyon_turevi, x0, iterasyon=10):

    x = x0

    for i in range(iterasyon):

        x = x - fonksiyon(x) / fonksiyon_turevi(x)

        print(f"{i + 1}. İterasyon: P{i + 1} = P{i} - f(P{i})/f'(P{i}) = {x}\n")

    return x

# Kullanıcıdan fonksiyonu alma

kullanici_fonksiyonu = input("Fonksiyonu giriniz (örneğin; x^2 için x**2, e^x için exp(x)): \n")

# Başlangıç değeri ve maksimum iterasyon sayısını alma

baslangic_degeri = float(input("Başlangıç değerini giriniz: \n"))

iterasyon_sayisi = int(input("Maksimum iterasyon sayısını giriniz: \n"))

try:

    x_simgesi = symbols('x')

    # Kullanıcıdan alınan fonksiyonu değerlendirmek için

    kullanici_fonksiyonu = eval("lambda x: " + kullanici_fonksiyonu)

except Exception as e:

    print("Hata: Girilen fonksiyonu değerlendirme sırasında bir hata oluştu.")

    print(e)

    exit()

try:

    # Kullanıcının girdiği fonksiyonun türevidir

    kullanici_fonksiyonu_turevi = diff(kullanici_fonksiyonu(x_simgesi), x_simgesi)

    print(f"Girilen fonksiyonun türevi: {kullanici_fonksiyonu_turevi}\n")

    kullanici_fonksiyonu_turevi = eval("lambda x: " + str(kullanici_fonksiyonu_turevi))

except Exception as e:

    print("Hata: Türev hesaplanırken bir hata oluştu.")

    print(e)

    exit()

try:

    # Newton-Raphson yöntemi çağrılır

    kok = newton_raphson(kullanici_fonksiyonu, kullanici_fonksiyonu_turevi, baslangic_degeri,
iterasyon_sayisi)

    print(f"\nSonuç: Kök = {kok}\n")

except Exception as e:

    print("Beklenmeyen bir hata oluştu.")

    print(e)
```

4.3 Pyhton Çıktısı:

```
Fonksiyonu giriniz (örneğin; x^2 için x**2, e^x için exp(x)):
exp(x) - sin(x) - 3*x
Başlangıç değerini giriniz:
0
Maksimum iterasyon sayısını giriniz:
5
Girilen fonksiyonun türevi: exp(x) - cos(x) - 3

1. İterasyon: P1 = P0 - f(P0)/f'(P0) = 0.333333333333333

2. İterasyon: P2 = P1 - f(P1)/f'(P1) = 0.360170713577634

3. İterasyon: P3 = P2 - f(P2)/f'(P2) = 0.360421680476020

4. İterasyon: P4 = P3 - f(P3)/f'(P3) = 0.360421702960324

5. İterasyon: P5 = P4 - f(P4)/f'(P4) = 0.360421702960324


Sonuç: Kök = 0.360421702960324


Process finished with exit code 0
```

```python
from sympy import symbols, diff, sin, exp, cos
def newton_raphson(fonksiyon, fonksiyon_turevi, x0, iterasyon=10):
    x = x0
    for i in range(iterasyon):
        x = x - fonksiyon(x) / fonksiyon_turevi(x)
        print(f"{i + 1}. İterasyon: P{i + 1} = P{i} - f(P{i})/f'(P{i}) = {x}\n")
    return x
# Kullanıcıdan fonksiyonu alma
kullanici_fonksiyonu = input("Fonksiyonu giriniz (örneğin: x^2 için x**2, e^x için exp(x)): \n")
# Başlangıç değeri ve maksimum iterasyon sayısını alma
baslangic_degeri = float(input("Başlangıç değerini giriniz: \n"))
iterasyon_sayisi = int(input("Maksimum iterasyon sayısını giriniz: \n"))
try:
    x_simgesi = symbols('x')
    # Kullanıcıdan alınan fonksiyonu değerlendirmek için
    kullanici_fonksiyonu = eval("lambda x: " + kullanici_fonksiyonu)
except Exception as e:
    print("Hata: Girilen fonksiyonu değerlendirme sırasında bir hata oluştu.")
    print(e)
    exit()
try:
    # Kullanıcının girdiği fonksiyonun türevidir
    kullanici_fonksiyonu_turevi = diff(kullanici_fonksiyonu(x_simgesi), x_simgesi)
    print(f"Girilen fonksiyonun türevi: {kullanici_fonksiyonu_turevi}\n")
    kullanici_fonksiyonu_turevi = eval("lambda x: " + str(kullanici_fonksiyonu_turevi))
except Exception as e:
    print("Hata: Türev hesaplanırken bir hata oluştu.")
    print(e)
    exit()
try:
    # Newton-Raphson yöntemi çağrılır
    kok = newton_raphson(kullanici_fonksiyonu, kullanici_fonksiyonu_turevi, baslangic_degeri, iterasyon_sayisi)
    print(f"\nSonuç: Kök = {kok}\n")
except Exception as e:
    print("Beklenmeyen bir hata oluştu.")
    print(e)
```

```
C:\Users\xrasi\AppData\Local\Programs\Python\Python312\python.exe C:\Users\xrasi\PycharmProjects\
Fonksiyonu giriniz (örneğin; x^2 için x**2, e^x için exp(x)):
exp(x) - sin(x) - 3*x
Başlangıç değerini giriniz:
0
Maksimum iterasyon sayısını giriniz:
5
Girilen fonksiyonun türevi: exp(x) - cos(x) - 3

1. İterasyon: P1 = P0 - f(P0)/f'(P0) = 0.333333333333333

2. İterasyon: P2 = P1 - f(P1)/f'(P1) = 0.360170713577634

3. İterasyon: P3 = P2 - f(P2)/f'(P2) = 0.360421680476020

4. İterasyon: P4 = P3 - f(P3)/f'(P3) = 0.360421702960324

5. İterasyon: P5 = P4 - f(P4)/f'(P4) = 0.360421702960324


Sonuç: Kök = 0.360421702960324


Process finished with exit code 0
```

KAYNAKÇA

[1] Nümerik Analiz, Ders notları, FATMA AYAZ

[2] Teorİ Teknik ve Uygulamalı Diferansiyel Denklemler, ADİL MISIR

[3] Udemy Python Sıfırdan İleri Seviye Programlama, Academy Clup

[4] Pyhton Dersleri (Türkçe) , Yakın Kampüs

[5] http://bumatematikozelders.com/

[6] https://acikders.ankara.edu.tr/